# Analysis of Reassignments for solving the Machine Reassignment Problem


Ollscoil na Gaillimhe
University of Galway

Piotr Bielski
School of Computer Science
University of Galway

*Supervisor(s)*
Takfarinas Saber

In partial fulfillment of the requirements for the degree of
*MSc in Computer Science (Artificial Intelligence - Online)*
2025

**DECLARATION** I, Piotr Bielski, hereby declare that this thesis, titled "Analysis of Reassignments for solving the Machine Reassignment Problem", and the work presented in it are entirely my own except where explicitly stated otherwise in the text, and that this work has not been previously submitted, in part or whole, to any university or institution for any degree, diploma, or other qualification.

Signature: _____

# Abstract

The reliance on cloud and high-performance computing platforms is becoming increasingly more widespread due to the rising demand for more computational resources across wide range of applications. However, the supporting infrastructure requires significant electrical power leading to negative impacts the environment. Additionally, at such large scales, inefficiencies in managing resources results in unnecessary and significant costs incurred by the compute providers and consumers. Therefore, to reduce the environmental impact and operational costs, the implementation of efficient resource management systems is essential. This report aims to investigate challenges in efficiently managing compute resources, researches optimization algorithms, specifically solutions for solving the ROADEF/EURO 2012 Challenge (Machine Reassignment Problem), common techniques and analysis the reassignments of best performing state-of-the-art solutions, and analysis their reassignment patterns to provide insights into the development heuristics for building better optimization algorithms.

**Keywords:** Machine Reassignment, Resource Management, Optimization, ROADEF, MRP

# Contents

# List of Figures

# List of Tables

# Chapter 1

# Introduction

High-performance computing and cloud platforms provide wide range of services in the form of Infrastructure-as-a-Service (IaaS), Platform-as-a-Service (PaaS), Data-as-a-Service (DaaS), which provide easy access to additional storage and compute resources. These services are in growing demand due to technological advancements in the recent years, resulting in explosion of resource-intensive scientific and industrial applications such as; artificial intelligence, SaaS enterprises, adoption of distributed systems across manufacturing and energy industries, as well as autonomous systems in the automobile industry - and all that with data centers as the backbone running this infrastructure. [1] The significant growth in demand for computational resources is pushing the construction on new large-scale data centers, with an estimation of Google having around one million servers. [2] While the benefits are plentiful across many industries, the data centers are requiring an increasingly larger supply of electrical power, with an estimation of already consuming around 1% of the global energy supply, while still being on the rise, [3] and having a growing impact on the environment due to CO2 emissions. [4] Resource management strategies are essential for efficient utilization of resources, which in turn reduce wastage of electrical power and operational costs.

However, cloud providers are facing significant difficulties in keeping up with the rate of growing demand to maintain efficiency and quality-of-service (QoS) standards due to the complexities of the dynamic nature of the environment, where the workload (e.g. network traffic) is not constant but instead fluctuates, requiring both intelligent and adaptive systems to optimally distribute workload among the available resources. [1] Considerable strategies have already been explored to improve the resource management efficiency, however, there is still much potential for further optimizations, and even small improvements can yield significant results at such scale. [3] In this context, Google presented a complex and large-scale machine reassignment problem (MRP) for the ROADEF/EURO contest in 2012 (https://www.roadef.org/challenge/2012/en/index.php) with the purpose of driving more efforts into research and innovation of novel optimization algorithms for tackling the major challenges of efficiently managing resources in dynamic environments.

The goal of this report is to analyze the behavior of multiple different best performing state-of-the-art optimization algorithms submitted for solving the ROADEF12 challenge by tracking the reassignment of processes during the lifecycle of the optimization in an attempt to provide new knowledge into what types of processes are reassigned by the different algorithms and to investigate whether there are any patterns in the reassignments or other characteristics that could be used as guideline for development of better resource management optimization algorithms.

## 1.1 Research Questions

The research project is guided by the following research questions:

1. RQ: What are the state-of-the-art algorithms for solving the machine reassignment problem?
2. RQ: What processes are reassigned by the top performing algorithms over the duration of the optimization process and are there any patterns in reassignments?
3. RQ: Can insights into the reassignments be used to create a framework for guiding the design of heuristics towards better selection of processes for reassignments?

These RQs guide the research process and should provide sufficient contextual background to understand the structure and approaches of optimization algorithms, as well as insights into whether there are any patterns that could be leveraged and guide the development of heuristics to guide the reassignment process in a more efficient manner.

## 1.2 Report Structure

- **Chapter 1: Introduction** lays out the foundation for the project, presenting the impact of growing reliance on compute resources, the challenges in efficiently managing these resources, and approaches in tackling the issue.
- **Chapter 2: Background** provides background information on the ROADEF/EURO 2012 Challenge (Machine Reassignment Problem), the problem definition, and common algorithms used for tackling the optimization problem, including information on reassignment and search strategies.
- **Chapter 3: Related Works** discusses the research objectives in more detail, the research methods applied when searching for related works, and an analysis of proposed solutions.
- **Chapter 4: Methodology** presents the research design and approach, specifically the required metrics for analyzing the evolution of solution over time, and the design of the tracking system to generate the dataset from reassignments.
- **Chapter 5: Experiments** contains all necessary steps to reproduce the experimental results, including the hardware specifications and system dependencies.
- **Chapter 6: Results** investigates the recorded reassignments, evaluates reassignments among solutions, and identifies reassignment patterns.

# Chapter 2

# Background

The following sections in this chapter provide background on the general topic and challenges of machine reassignment, an introduction to ROADEF/EURO annual contests, expanded details of the specific challenge presented in 2012 - the Google Machine Reassignment Problem ("GMRP"), description of the problem definition that defines the various components of the problem, and finally information on common algorithmic approaches for solving the MRP.

## 2.1 Machine Reassignment Problem

**Machine Reassignment Problem (MRP)** is a complex combinatorial optimization problem that involves the distribution of workload across a set of machines in a way to optimize the utilization of available resources on the machines, and is particularly relevant in dynamic environments, where workloads fluctuate and available resources also change over time, such as in cloud computing and HPC platforms. The fundamental action in MRP is the reassignment of processes from one machine to another with the goal of improving overall system performance and efficiency.

However, simple resource management systems struggle to adapt to these changes effectively, and through controlled reassignments using intelligent algorithms are needed to improve efficiency of the system.

However, the major challenge stems from the complexity of the problem - which requires balancing many variables, of which values may change, as is the case in cloud computing where workloads are continuously received or completed resulting in the conditions always changing, and such, not only it is insufficient to run the optimization algorithm once to distribute the workload, but instead it is an on-going iterative process which involves constant evaluation of the system and potentially cycles of steps of reorganizing the processes across machines based on current workloads and available resources, but also given the significant number of variables necessary for consideration, e.g. resources required by the process (RAM, CPU, DISK, etc.), available resources on each machine, the total capacity of each machine, or balancing load across machines - the MRP problem is defined as NP-hard. This classification implies that finding an optimal solution is computationally not feasible, especially for instances of realistic size and complexity, and hence often requires the use of heuristic and metaheuristic approaches to tackle these types of problems. Such approaches aim to find high-quality solutions in a reasonable timeframe, but with the trade-off of no guarantee for obtaining global optimality. [5]

## 2.2 ROADEF/EURO 2012 Challenge - Google Machine Reassignment Problem

The ROADEF/EURO contest is jointly organized by French Operational Research and Decision Aid Society (ROADEF) and Association of European Operational Research Society (EURO) and appears regularly since 1999.

The challenges are open to everyone and aim to promote innovation and collaboration between researchers. Generally, participants are split into two categories; (1) Junior category for young researchers, and (2) Senior category for experienced and qualified researchers. The challenges provide an opportunity to tackle complex industrial optimization problems and have been received as widely successful. [6]

As for the specific challenge related to this report, Google presented a formulation of the machine reassignment problem for the ROADEF/EURO 2012 contest, called the Google Machine Reassignment Problem ("GMRP"), [1] with a set of benchmarking instances to the research community. The problem is commonly encountered in dynamic environments such as cloud computing, high-performance computing platforms, data centers, and other similar large compute platforms, where workloads can fluctuate and it is beneficial to reallocate available resources to where they are needed based on current demand.

Once published, 82 teams registered for the GMRP contest. The first stage involved the participants to submit their initial solutions to be evaluated against the smallest dataset, "A", of which, only 30 teams could qualify into the next round. In the second round, the qualified teams then moved to more realistic set of instances made available to them, called dataset "B", to allow for fine-tuning their algorithms and prepare for having their solutions run against the final, largest, and unknown dataset "X".

Overall, the GMRP contest created a competitive environment that resulted in a diverse array of novel state-of-the-art approaches and highlights the problem's profound relevance in the industry. The results and algorithms from the contest offer rich dataset for the analysis of efficacy of different algorithmic strategies and other explored domain-specific heuristics that pushed performance of the

[1] https://roadef.org/challenge/2012/en/

solutions even further beyond their original capabilities.

## 2.3 Algorithmic Approaches for Solving MRP

At its core, reassignment is the fundamental action of changing the allocation of a process from one machine to another, and is also the only action needed to improve the efficiency of the larger system through optimization towards better utilization of available resources, provided the reassignments are controlled using an intelligent optimization algorithm. Given it is not feasible to run the optimization process for the entire problem at once, a common approach involves reducing the scope by considering only a subset of processes to optimize through reassignments, and iteratively repeating this procedure each time selecting another subset of processes. For such algorithm, the two primary components are; subproblem selection and search mechanisms - these define how the algorithm determines which subset of processes is selected in each iteration and how it explores the solution space through possible reassignments for the current subset of processes. This approach is the basis for two fundamental search paradigms used to tackle the MRP; local search and large neighborhood search. [7]

### 2.3.1 Levels of Abstraction in Optimization Approaches

Algorithms are generally categorized based on their approaches [5] and include:

- **Heuristic** approaches are simple, problem-specific rules designed to quickly find "good enough" solutions and rely on domain knowledge of the specific problem, e.g. in context of MRP, it could be a simple rule to always first assign the largest process to the least loaded machine. The purpose of such approach is most beneficial in situations where speed is prioritized over optimality.

- **Metaheuristic** approaches are higher-level algorithmic frameworks designed to guide the underlying heuristics or search methods. These provide additional capabilities for escaping local optima and are capable of efficiently exploring large solution spaces.

- **Hyper-heuristic** approaches operate at even higher level than metaheuristics, and are capable of automatically generating appropriate heuristics or metaheuristics to solve wide range of problems, however, also comes at much higher computational cost as compared to the other approaches.

### 2.3.2 Common Search Methods for solving Optimization Problems

#### 2.3.2.1 Local Search

**Local Search (LS)** method optimizes the objective cost function through an iterative process of small, incremental improvements. LS starts from an initial solution and explores the solution space by applying small modifications, known as *move operators*, to the current solution. Basic move operators include actions such as process reassignment ("Shift") and swapping the assignment of two processes ("Swap"). Repeated application of these move operators on different parts of the solution generates a set of possible, "neighboring" solutions. Once the solution space is sufficiently explored, the second component of LS is the *search strategy* to determine the move to accept; *Steepest descent strategy* generates and evaluates the entire neighborhood, and moves to the solution with the best improvement - providing better results at the cost of higher computational cost, whereas a *Greedy strategy* generates neighboring solutions one-by-one and immediately accepts the first solution that shows improvement, even if marginal. The simplest form of a local search method is known as the **First-Improvement**

**(Greedy) Hill Climber**, and consists of an LS method using the Greedy search. [8]

#### 2.3.2.2 Large Neighbourhood Search

**Large Neighborhood Search (LNS)** is a metaheuristic that extends LS, and consists of two primary components: Destroy phase and Repair phase. In the first step, Destroy phase, some section of the solution is "destroyed", which involves iterative selection of subset of processes, followed by having them unassigned - for the purpose of *neighborhood exploration*. The next stage, Repair phase, involves rebuilding the unassigned processes while optimizing for the cost function and often includes exact mathematical optimization algorithms, such as Constraint Programming (CP) or Mixed Integer Programming (MIP), to find the optimal reassignments for the current process subset. While these exact solver methods can guarantee optimal solutions, they come with the trade-off of being computationally expensive due to their exhaustiveness. [9] **Variable Neighborhood Search (VNS)** is a sophisticated variation of LNS, which follows similar processes of Destroy and Repair phases but with the addition of a diversification strategy that expands the explored solution space by changing the type and size of neighborhood during the search process, increasing the potential for finding a better solution and improved ability to escape local optima. VNS is also capable of applying intensification strategies to narrow down the search towards specific neighborhoods through the adjustment of subproblem selection by using smaller neighborhoods. [10]

# Chapter 3

# Related Work

Related Work chapter provides an overview of the approaches proposed for solving the Google Machine Reassignment Problem, including their underlying core algorithms, search mechanisms, and heuristics, and lastly a systematic review of all submitted solutions for the ROADEF/EURO 2012 challenge.

## 3.1 Literature Research Objectives

Objectives for the literature review are guided by the research questions to provide an overview on the various state-of-the-art approaches submitted in the ROADEF/EURO 2012 challenge to solve the GMRP problem, novel problem-specific heuristics, and any other techniques to improve the performance of the algorithms. The search also includes more recent works that have been published since the contest, which have used the GMRP datasets for benchmarking and evaluation of their proposed algorithms to investigate the evolution of the approaches over time. Additionally, the review of algorithmic approaches and their ranking in the contest serves as source of suitable candidate algorithms for selection into the analysis of their reassignments.

## 3.2 Literature Research Method

The literature research was performed primarily on Google Scholar, NUIG James Hardiman Library, and Scopus for documents containing keywords; "Machine Reassignment Problem", "Google Challenge 2012", "Resource Management", "ROADEF12", and "Resource Optimization".

Abstract sections of the resulting documents from the search queries were scanned and selected for in-depth analysis depending on their relevance to the topic. The query was further narrowed down to include peer-reviewed papers in high-quality journals (Q1 and Q2) that were published from the year of the publishment of the Google Machine Reassignment Problem, 2012 and onwards. Furthermore, of the selected relevant papers, their references and citations were investigated to provide more informational background and potential implications on the research that followed. However, given the niche nature of the topic, the search criteria was more relaxed towards papers published by the participating teams of their own GMRP approaches.

## 3.3 Proposed algorithms for solving Google Machine Reassignment Problem

The best performing algorithm in the ROADEF/2012 GMRP contest was presented by team S41 consisting of a variable neighborhood search method including two specialized move operators, heuristics to prioritize load cost reduction during process selection, and a technique for "random" noising of the objective function to temporarily modify the original weighted sum, to move the search out of local optima. [11] In second place came in team S38 using an LNS-based algorithm and a CP exact solver to compute the optimal reassignments in the

context of the selected subset of processes, as well as heuristics for guiding the search and subproblem optimization. [12] Team J12 placed in third place, and on the first place within the junior category, with an algorithm using a hybrid approach of switching between heuristic and metaheuristic; HC and MIP-LNS. The algorithm initializes with HC and keeps running it until hitting a local optima, falls back to MIP-LNS, and returns to running HC again. [13] In [14], a hyper-heuristic approach is proposed to allow the algorithm working with different neighborhoods and have the ability to select the most suitable heuristic, and is based on Simulated Annealing (SA) using two low-level heuristics 'H0' and 'H1', where H0 targets the exploration of larger neighborhoods and used as supporting heuristic for H1 in situations where the search is stagnating and current candidate solution is significantly worse. Iterated Local Search (ILS) was presented by team S27 [15] for which both randomized perturbations to perform random shifts where evaluated against an integer programming exact solver (IP-ILS). The results demonstrated significant performance difference of the IP solver over randomized perturbations, and the final proposed algorithm used IP-ILS as the core component. A multi-start iterated local search is proposed in [16], which repeatedly runs a local search algorithm from different initial solutions and only focuses the optimization process on load and balance costs. Neighborhood exploration is done using basic move operators; Shift and Swap, and two other additional specialized operators; (1) 'Home Relocate' to randomly select a number of processes that are currently assigned to a machine different to their initial machine and reassigned back to their initial machine, (2) 'K-Swap' randomly selects K pairs of machines and swaps random groups of processes between them. In [17], proposed an approach originally designed for solving the classical bin packing (BP) problem - the vector bin packing method. It's capabilities are expanded along with many diverse heuristics to efficiently tackle the GMRP problem.

Later works, post the ROADEF 2012 contest, have introduced more sophisticated techniques as well as refined on the simpler approaches, such as presented in [18]. The approach consists of a simple local search using steepest descent strategy with basic move operators. Due to the parameter-free design of LS, it was used as the basis for the evaluation of a new heuristic that combines the existing neighborhoods, and then randomly selects one from the list to generate a neighborhood solution during the search process - resulting in better performance as compared to Shift, Swap, and BPR operators alone.

Additionally, due to the success of the contest, new algorithms have been proposed and evaluated using datasets from the GMRP problem for many years following the ROADEF 2012 contest, including techniques such as capable of outperforming all of the submitted algorithms in the contest. [19]

Systematic review of approaches [20] shows the overall performance rankings being predominantly dominated by VNS-, LNS-, and Ant-HH- based algorithms by providing sufficient flexibility in their designs to explore the solution space, and capable of escaping local optima. Additionally, use of MIP exact solvers is more common over CP solver as shown in [6], but with the CP solver based algorithms scoring higher.

# Chapter 4

# Problem Definition

The problem definitions [2] outlined in the following sections are fundamental in understanding the key concepts of GMRP which are at core of all optimization algorithms, and provides the necessary context later interpretation of results during the analysis of reassignments. These definitions are formulated through three interacting components; decision variables, constraints, and objectives - and collectively define the available information to the algorithms, set out boundaries on the feasible solution space that can be explored during the search process, as well as the objectives that act as implicit drivers for how the solution space is navigated through reassignments. Additionally, the definitions often serves as the basis and source of many domain-specific heuristics that are often implemented alongside the core algorithms to further improve their performance.

### 4.0.1 Decision Variables

The decision variables represent the choices available to the algorithm, i.e. the main decision is to determine where each process "should" run and capture the reassignments. At any valid solution, each process is assigned to exactly one

[2] https://roadef.org/challenge/2012/en/

machine, which translates to the current assignment of processes to machines.

- **Machines**, $\mathcal{M}$, refers to set of machines used for the execution of processes, where each machine, $m \in \mathcal{M}$, has a set of predefined and finite amount of resources such as CPU, RAM, DISK.

- **Processes**, $\mathcal{P}$, refers to a set of processes which are the fundamental units of work. Each process, $p \in \mathcal{P}$, has specific resource requirements (CPU, RAM, DISK) and consumes these resources off the machine to which it is assigned, $\mathcal{M}(p) = m$.

### 4.0.2 Constraints

Constraints define rules that set boundaries for the feasible solution space to which algorithms must adhere and thereby have an effect on possible assignments. These constraints reflect real-world physical limitations such as available computational resources on servers, process conflicts and dependencies, load distribution mechanisms, and costly migration of processes over distributed networks.

- **Capacity Constraints**: A process cannot be assigned to a machine if the machine does not have enough available resources, $\mathcal{R}$. The machine must have enough capacity of all resources (CPU, RAM, DISK) to host the process. The capacity of resource is defined as $\mathcal{C}(m, r)$.

- **Conflict Constraints**: Defines that processes belonging to same service must run on distinct machines to avoid conflicts, and does not allow processes of the same service to run on the same machine, where a service, $s \in \mathcal{S}$, consists of processes.

- **Spread Constraints**: Defines the minimum number of locations, *minSpread*, over which processes of a service must be distributed, where *loca-*

*tion*, $l \in \mathcal{L}$, consists of a set of machines.

- **Dependency Constraints**: Services can have a dependency on another service. In such cases, processes of the dependent service must run in the neighbourhood of the other service processes. Here, a neighbourhood is defined as a set of machines as well, $n \in \mathcal{N}$.

- **Transient Usage Constraints**: While a process is migrated from one machine, $m$, to another, $m$', the resources required by the process are temporarily consumed on both machines. Eventually, once the migration is complete, the resources are freed from the initial machine.

### 4.0.3 Objectives

Objectives are the variables used by algorithms for guiding the optimization process and defines incurred costs from certain actions, but also penalties for violating certain conditions:

- **Load Cost** is incurred for over-utilizing machine resources beyond predefined safety capacity threshold, and serves as penalty for pushing machine usage beyond preferred operational load, even if still within the absolute physical limits. Each machine defines a safety capacity per resource, $\mathcal{SC}(m, r)$, over which the penalty is incurred.

- **Balance Cost** defines an objective to encourage maintaining a balance between available resources across machines and incurs a penalty for imbalances.

- **Process Move Cost** accounts for the overhead of moving a process from one machine to another and incurs a cost for each process reassignment.

- **Service Move Cost**: Designated to balance the number of moved processes across different services. It is defined as maximum number of moved processes for any single service.
- **Machine Move Cost**: Cost of migrating any process to another machines. The cost varies depending on specific source and destination machines.
- **Total Objective Cost**: Weighted sum of all individual costs.

In summary, the **Total Objective Cost** is an aggregation of individual cost components, and effectively transforms the problem from multi-objective into a single-objective optimization problem.

# Chapter 5

# Methodology

This chapter outlines the research design and approach for the analysis of reassignments, including the selection of datasets, algorithms, and metrics for tracking the evolution of solutions over time.

## 5.1 Overview of Design Process and Approach

The overall idea of the project is to analyse reassignments performed during the optimization process of state-of-the-art algorithms to analyse their behaviour and identify any patterns in the reassignments that could be used to guide the design of heuristics for better selection of processes for reassignment.

Therefore, the initial set of requirements to perform the analysis includes:

1. Initial assignment dataset,
2. Selection of algorithms for analysis,
3. Dataset of Reassignments

The first two requirements focus on the initial setup and the steps in order

to obtain the third step of having a dataset of reassignments, and are further expanded in the following sections.

## 5.2 Initial Dataset: Process-to-Machine Assignments

The initial dataset of process-to-machine assignments is available on the ROADEF/EURO 2012 challenge page, as well as a solution checker, and the official problem definition.

As previously mentioned, the challenge consists of three datasets; A, B, and X, that vary in size and complexity, where dataset A is the smallest and simplest, dataset B is larger and more complex, and dataset X is the largest and most complex. Furthermore, each dataset consists of 10 instances of varying size and gradual increase in complexity - though still within the boundaries of the dataset characteristics.

Only one dataset, A, is used for the analysis of reassignments as it contains a smaller set of instances, allowing the algorithms to complete more reassignments as compared to the larger and more complex datasets, B and X. Given the underlying logic for reassignments remains the same, it is assumed the reassignment behavior should also remain the same, whilst at the same time generating a larger dataset of reassignments is preferable.

Overview of datasets and the specific variables of the two relevant dataset A instances [1] are:

Table 5.1 provides an overview of the dataset characteristics in the contest and the two dataset A instances used in the analysis, where "[100, 1000]" defines a range of possible values for instances within the dataset, e.g. minium of 100

[1] https://roadef.org/challenge/2012/files/Roadef%20-%20results.pdf

| Variable | Dataset A | Datasets B and X | Instance a1_1 | Instance a1_4 |
|---|---|---|---|---|
| Processes | $[100, 1000]$ | $[5000, 50000]$ | 100 | 1000 |
| Machines | $[4, 100]$ | $[100, 5000]$ | 50 | 1000 |
| Resources | $[2, 12]$ | $[3, 12]$ | 2 | 3 |
| Transient | $[0, 4]$ | $[0, 4]$ | 0 | 1 |
| Services | $[10, 125]$ | $[500, 1000]$ | 10 | 100 |
| Neighbourhoods | $[1, 50]$ | $[5, 5]$ | 1 | 50 |
| Locations | $[1, 50]$ | $[10, 100]$ | 4 | 50 |
| Dependencies | $[0, 10]$ | $[30, 70]$ | 10 | 10 |
| Balance | $[0, 1]$ | $[0, 1]$ | 1 | 1 |

Table 5.1: Overview of dataset characteristics and the relevant dataset A instances

and maximum of 1000.

Therefore, a11 and a14 instances used and used as the starting point for the initial assignments, as they represent the smallest and largest instances within dataset A, and should provide a good representation of the reassignment behavior across the entire dataset.

## 5.3 Selection of Algorithms for Analysis of Reassignments

Given the prominence of VNS and LNS-based algorithms in the contest, as well as their continued use in later works, and overall strong performance - these are selected as the core algorithms for the analysis of reassignments.

The best performing implementations of these algorithms based on GMRP results are selected for the analysis of reassignments, and include:

- VNS proposed by team S41 [11]
- CP-LNS proposed by team S38 [12]
- HC-(MIP-LNS)-HC proposed by team J12 [13]

VNS-based approach incorporates two specialized move operators, along side the two basic operators, used for the exploration of the neighborhood; `Chain` involves triggering a sequence of interdependent reassignments, such as cyclic exchanges, with the purpose of further expanding the exploration space `BPR` (Big Process Reassignment) specifically targets the reassignment of large processes by removing multiple of processes at once, as these are generally the hardest to reallocate, and improves the efficiency of the other three operators (Shift, Swap, Chain) that often struggle with reassignment of the larger processes. BPR is also attributed the highest potential of contribution towards the improvement of objective cost. Additionally, the search is guided by heuristic to sort processes by size and prioritize the reassignment of large processes at first, as it becomes harder to do so in later stages, and explores the neighborhood using move operators in the following order; BPR, Shift, Swap, Chain. Another technique incorporated into the algorithm aims to expand the diversification strategies and consists of "noising" the objective function, which temporarily increases the weight cost of a constrained resource, and then optimizing for the noised objective function, forcing the search out of local optima. The use of specialized BPR operator as well as process size heuristic indicates this approach could be a good candidate to extract meaningful information based on reassignments,

In [12], both MIP and CP solvers were evaluated with an LNS-based approach. For MIP, performance gains were observed in the selection of all process from small number of machines over selection of few processes from a large number of machines. However, CP has shown better performance on larger instances and was selected as the solver for the final submission. Additionally, significant gains that pushed the base CP-LNS performance are attributed to multiple domain-specific heuristics at different layers of the algorithm. Specifically, cost-driven heuristic is used to guide the search by prioritizing machines with the highest

current cost to be used for the subproblems, mixed with some machines selected at random - with in this approach, only a subset of processes of the selected machines is considered for reassignment to reduce probability of situations where a process is assigned back to the same machine from which it was just unassigned. The variable domains of CP are also updated using heuristics to improve performance of subproblem optimization by reducing the search space; (1) *variable ordering heuristic* are based on aggregation of potential improvement of objective cost, the total weight of process requirements, as well as the available machines for the process, (2) *value ordering heuristics* define the selection of suitable machine for a process based on minimum cost, and (3) *filtering rules* to narrow down the solution space by removing machines from consideration that would produce infeasible solutions. Based on the heuristics in this approach, it is difficult to make assumptions on any expectations for potential patterns in reassignments as the cost-driven heuristic does not target processes of specific size, but instead machines of specific cost - where even knowing a machine has high cost, an indication of containing hosting something heavy, could mean it is either many small processes or few large processes.

Even though MIP solver was observed to provide suboptimal performance on larger instances based on investigations of the previous approach, the HC-LNSHC algorithm is capable of delivering solid performance with the MIP solver due to its architectural hybrid design consisting of a heuristic and a metaheuristic working together; (1) Local Search with Greedy search, i.e. First-improvement Hill Climber (FI-HC), and (2) LNS using MIP for subproblem optimization. Rather than relying on the computationally expensive exact solvers on each iteration, the algorithm instead initializes with FI-HC to rapidly iterate towards any improvement in the objective cost as fast as possible. The iteration speed is further optimized with an "Optimistic Improvement" heuristic for subproblem selection

based on estimation of the potential for cost reduction derived from load and move costs, and then prioritizing processes with the highest potential - rather than having to do the more expensive computation of entire solution cost. In addition, the FI-HC also maintains a tabu list of the previously explored machines to avoid cycling. The FI-HC is left running until eventually reaching getting stuck in a local optima, from which if it is unable to find further improvements, the algorithm switches to MIP-LNS. The MIP-LNS also uses heuristics for subproblem selection to prioritize subsets based on potential to reduce the objective cost, while the subproblem size is adaptively adjusted by starting off small, and iteratively increasing the subproblem size if it is optimally solved until reaching a point where the optimization fails, at which point the previously solved subproblem is accepted as the best. After the MIP-LNS completed, the algorithm falls back to the FI-HC again and repeats the process. Based on the analysis of the approach, the performance can be attributed to the overall design of the algorithm and the effectiveness of heuristics incorporated into both the FI-HC and MIP-LNS. It is possible the use of MIP on larger instances is also resulting in similar performance hits when ran that were observed in previous approach, but the FI-HC could be minimizing the impact through its efficient and fast iterations.

This in-depth analysis of the algorithms provides valuable information on the state-of-the-art algorithms, whose performance competed against over 80 other participants during the ROADEF 2012 contest, and emerged as the top three best performing optimization algorithms. Key insights help to address RQ1, seeking to answer on the best performing algorithms and their characteristics - which includes all three algorithms having incorporated some level of domain-specific heuristics that provide significant performance gains well above original capabilities at various layer to guide the algorithm as it navigates the solution space. Also, the incorporation of randomness is common to allow the search

espace from getting trapped in a local optima.

## 5.4 Metrics for the analysis of reassignments

Of the selected algorithms, none of them generate datasets of reassignments by default and therefore changes are required for tracking the reassignments. Therefore, this leads to two additional requirements:

1. Design and pplanning of needed reassignment metrics
2. Implementation of tracking into the algorithms to generate the dataset

The tracked metric for each reassignment should provide sufficient information for the analysis and include:

- **Process ID**: Serves as the primary ID to identify the process that is being reassigned at the current step
- **Process Size**: Provides information on the aggregation of process resource requirements to capture insights into what types of processes are being reassigned, how frequently, and potential patterns.
- **Solution ID**: Allows to derive further information on how the solution evolves over time such as the number of reassignments performed in the process of moving from the current solution to the next solution accepted by the search mechanism.
- **Total Objective Cost**: Associates reassignments and their potential quality through the impact on the objective function and should allow to track the evolution of objective cost over reassignments.
- **Improvement Cost**: This is the delta between the objective cost between two adjacent reassignments. It is primary tracked for convenience to provide insights into the direction and size of improvement in objective cost for

each reassignment, to tackle questions such as whether the reassignment of processes of specific size (large vs. small) has greater potential.

## 5.5 Tracking System Design

The goal of the tracking system is to provide the ability of capturing reassignments and achieved by modifying the original codebases of each algorithm and have them write the reassignment metrics to a file each time a reassignment is performed. This approach is not optimal in terms of algorithm performance, as writing to file and especially this frequently for every assignment, is an expensive operation and will slow down the algorithms. However, the approach is still suitable for the analysis as the performance impact does not affect the types of reassignments, just their number - even with less recorded reassignments, it is more than enough.

Steps taken in the planning and design of the tracking system:

1. Clone repositories containing the algorithms,
2. Run the algorithms before making any changes to ensure they are working as expected,
3. Investigate structure of the project to understand the implementation of the algorithms,
4. Design an approach for the implementation of the tracking system,
5. Modify the original codebases to capture reassignments,
6. Run the algorithms using datasets of initial assignments and capture the reassignments,
7. Finally, use the newly generated reassignments dataset for the remaining analysis.

Source code for the original algorithms was found at:

- S41 (VNS): https://github.com/harisgavranovic/roadef-challenge2012-S41
- S38 (CP-LNS): https://sourceforge.net/projects/machinereassign
- J12 (HC-LNSHC): https://bitbucket.org/wjaskowski/roadef-challange-2012-public

The modified codebases to track reassignments along with the analysis are available under one repository: https://github.com/circleorange/mrp-approach-analysis

## 5.6 Tracking System Implementation Challenges and Limitations

The first codebase investigated was of the HC-LNS-HC algorithm, developed in Java. The main difficulty, and a common theme across all three codebases, is the lack of information on the commands or arguments required for running the projects. The LNS is using MIP solver, which at the time of original implementation was CPLEX Solver v12.5 - however, only the latest version is available for download, which resulted in the requirement for updating the dependency to CPLEX v22.1.1, and along with it multiple additional changes to the codebase as the change new CPLEX version is not backwards compatible. Major challenges were encountered in the following two codebases of CP-LNS and VNS algorithms, primarily due to all algorithms being implemented in different programming languages (Java, C, C++), preventing the reusability of code for tracking assignments and instead requiring individual implementations each time. Additionally, the CP-LNS algorithm is implemented in a way that makes retrieving much of the reassignment metrics not possible - such as the process size which in the end required post processing steps on the captured reassignments to populate the

column using dataset generated by one of the other algorithms, since all algorithms were ran using the same instance. Secondly, the approaches are lacking the concept of "Solution ID" and was not feasible to be tracked, as well as performing both the neighborhood exploration and accepted reassignments in single `for` loop, making the tracking of actual reassignments very difficult. Unfortunately the inclusion of explored reassignments during neighborhood exploration is not feasible as it generates a reassignments file of over 16GB and crashes the Python kernel, and required significant efforts to complete the implementation while ensuring exploration moves are not tracked. The tracking implementation in the CP-LNS codebase is questionable to some extent as it relies on flipping a flag back and forth in an attempt to filter out exploration moves and only capture true reassignments.

# Chapter 6

# Experimental Setup

## 6.1 Hardware specifications and system dependencies

The algorithms were executed on the same hardware of following specifications:

- 13th Gen Intel Core i9-13950 HX with 24 cores (32 threads) and max turbo frequency up to 5.50 GHz.
- 192 GB DDR4 RAM @ 3600 MT/s

In addition, the following system dependencies are required to support the execution of investigated algorithms:

- Ubuntu 22.1 - but should also work without changes on other UNIX-based systems and will require changes to run on Windows OS as some build scripts are written in bash and would need to be converted to PowerShell.
- JDK with Java 8 support
- Apache Ant
- IBM ILOG CPLEX Studio v22.1.1 (specifically for the MIP solver)

- GCC C++ compiler with C++11 support
- Make
- Python 3.x

## 6.2 Streamlined pipeline to run the algorithms

VS Code was used in both the implementation and analysis stages of the project as it provides support for all necessary tools and technologies used through the investigation.

During the implementation stage, a more streamlined pipeline was developed for testing and running the algorithms with the use of "Run Tasks" feature provided in VS Code. The feature allows to define set of configurations in a `tasks.json` file such as the commands to build and run the algorithms, as well as to pass in any additional arguments. These tasks resulted in significant improvement in convenience throughout the investigation as the algorithms are implemented in different programming languages, and hence require different commands to run, but also multiple arguments need to be passed into the program including; path to initial assignments dataset, path to file containing data on the problem variables (constraints), output file, and other project specific arguments. With the current pipeline, any algorithm can be executed by simply opening the Command Palette in VS Code ¿ Run Tasks and selecting the specific task to execute, e.g. "HC-LNSHC: Run Full Duration Analysis (a1_4, 300s)" or "HC-LNSHC: Test Run (a1_1, 10s)" to either run the algorithm for the full 5 minute duration or a quick test run for 10 seconds on a different, easier dataset that allow for faster development iteration cycles throughout the implementation and testing of the changes for tracking reassignments.

# Chapter 7

# Results

This chapter presents the results of the experimental runs conducted to evaluate the selected algorithms, with the primary focus into the analysis of reassignments made by the algorithms during their execution. Once the plots are introduced, they are discussed in detail to highlight the key findings and observations.

| | VNS | CP-LNS | HC-LNSHC |
|---|---|---|---|
| # Reassignments | 4344 | 25440 | 330 |
| Avg. Process Size Moves | 212k | 37k | 268k |
| # Big Process Moves | 45 | 9 | 4k |
| # Neighbourhoods | 8 | 6130 | 330 |
| Avg. Moves/Neighborhood | 543.0 | 4.15 | 1.0 |

Table 7.1: Optimization characteristics of reassignments.

Table 7.1 provides an overview of the optimization characteristics of reassignments, where '# Reassignments' is the total reassignments count made by each algorithm to reach their final solution, 'Avg. Process Size' represents the average size of the processes involved in the reassignments, '# Big Process Moves' counts the significant moves made during the reassignments (threshold to qualify process as BPR is based on an arbitrary threshold of 2e6 used in the analysis to provide common ground for comparison as otherwise it would very across algorithms),

‘# Neighbourhoods‘ shows the number of neighborhoods traversed from the initial state of assignments to the final solution, and ‘Avg. Moves/Neighborhood‘ provides the average number of moves per neighborhood.

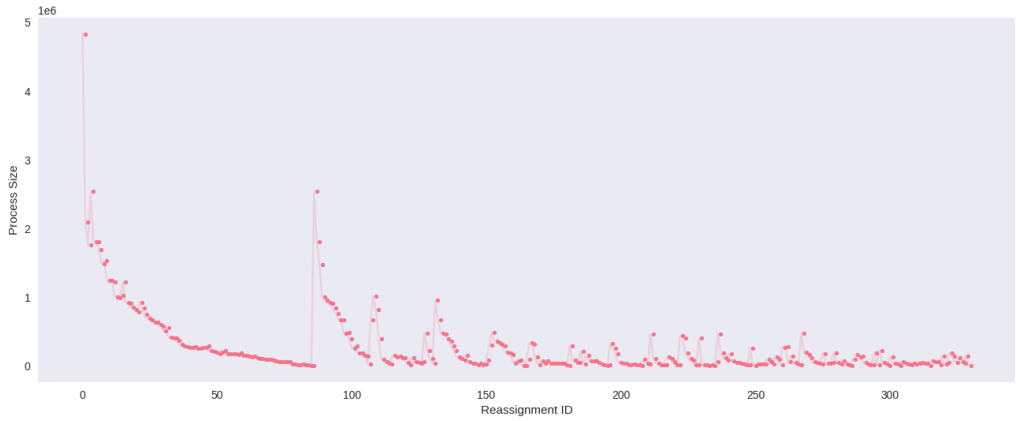


Figure 7.1: HC-LNSHC Algorithm Process Sizes of each Reassignment

7.1 presents sizes of the processes that are being reassigned over time by the HC-LNSHC algorithm during its execution on the a1_4 instance to provide insights into the optimization process of the underlying algorithm.

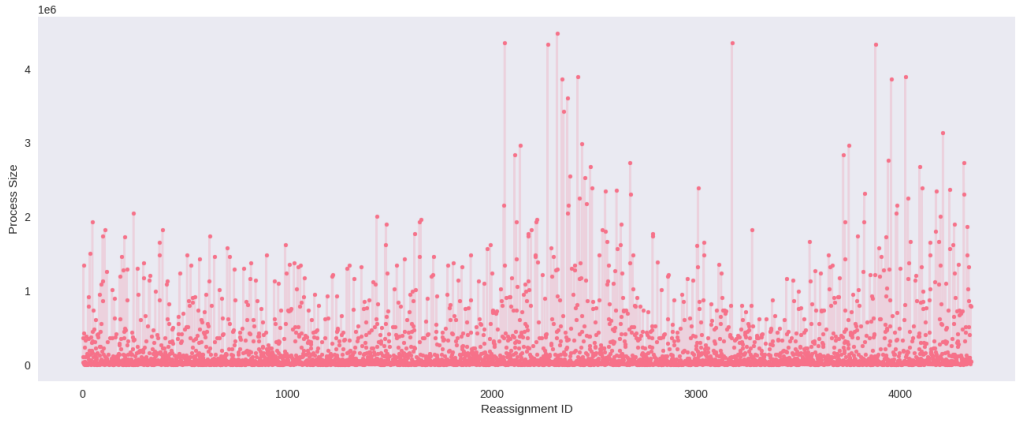


Figure 7.2: VNS Algorithm Process Sizes of each Reassignment

Similarly, Figure 7.2 illustrates the reassignments made by the VNS algorithm during its execution on the a1_4 instance.

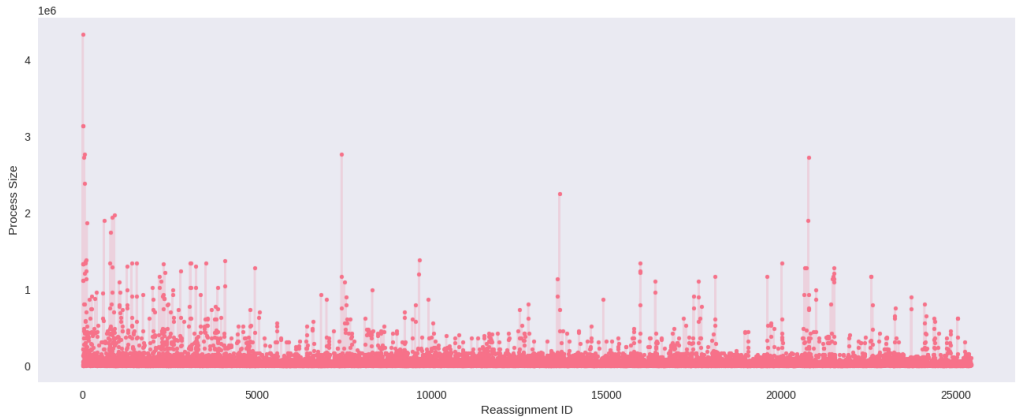


Figure 7.3: CP-LNS Algorithm Process Sizes of each Reassignment

Lastly, the reassignments made by the CP-LNS algorithm during its execution on the a1_4 instance are illustrated in Figure 7.3.

Based on the results of the reassignment, HC-LNSHC presents the clearest signs of patterns in the reassignment processes, Figure 7.1. Specifically, the prioritization of large processes early, and as the optimization process evolves, it becomes more difficult to find suitable reassignments, leading to the gradual decrease in process size over iterations, until the algorithm reaches some critical threshold causing an immediate spike, and then continuing the same gradual downwards trend as previously, with each subsequent spike being less pronounced and shorter than the previous.

Further analysis into the objective cost over reassignments, Figure 7.4, reveals that each spike in process size corresponds to a spike in the improvement of the objective function, which aligns with the algorithmic design of the HC-LNSHC presented in [13], where the gradual improvements indicate the optimization of the first-improvement hill-climber, delivering small incremental improvements on each iteration, all the way until eventually getting stuck in a local optima.

This would be the point at which the algorithm switches to MIP-LNS in order to trigger a larger reassignment process in an attempt to escape the local optima,

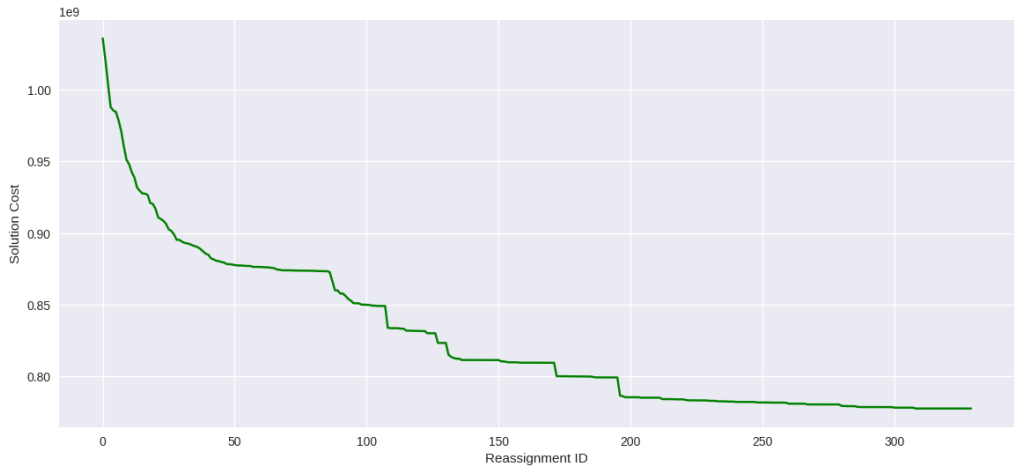


Figure 7.4: HC-LNSHC Algorithm - Objective Cost over Reassignments

thereby producing a larger improvement gain and resulting in the spikes caught by the tracked reassignment metrics.

Interestingly, based on the results in Table 7.1, the HC-LNSHC algorithm was capable of producing a strong solution while having an average of only 1 reassignment between neighborhoods and 330 reassignments, as compared to 4344 reassignments by VNS and 25440 by CP-LNS, showcasing effectiveness and efficiency of the FI-HC and LNS combination in rapidly navigating the solution space without having to perform frequent exhaustive searches.

Another insight from the HC-LNSHC analysis, by combining all the observations, is the relation between the reassignment of single larger process and a larger jump in improvement gains, supporting the idea of larger processes carrying more potential and builds on the knowledge towards both RQ2 and RQ3 for potential deisgn of a heuristic to guide the search process based on these reassignment patterns. So far, the findings suggest selection of the larger processes for reassignments, followed by selection of the next largest process in next iteration (which based on the reassignments appears to nearly always be smaller), and so on, until reaching a point where the algorithm is unable to find any further

improvements with a process of size that is smaller than the previous.

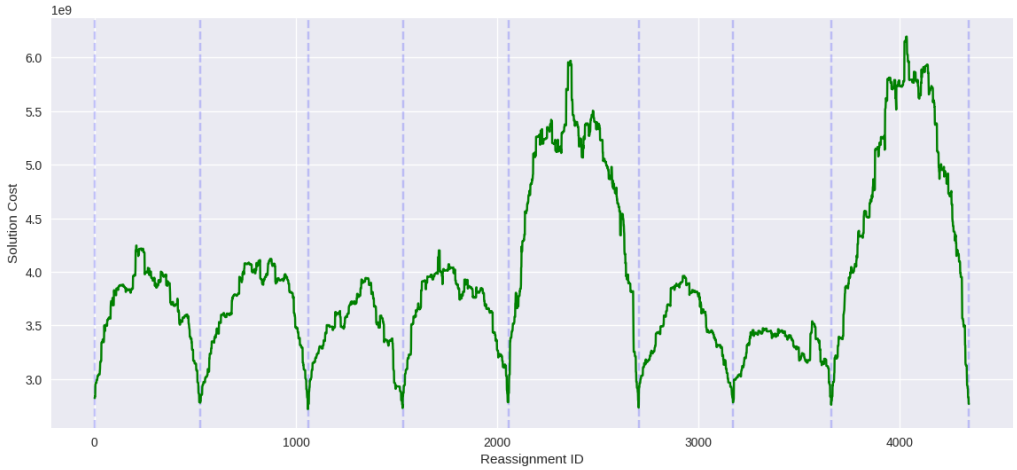


Figure 7.5: VNS Algorithm - Objective Cost over Reassignments

The other two algorithms, VNS and CP-LNS, do not exhibit the same level of correlation between process size and improvement gains or patterns in their reassignments. VNS, for instance, tends to make more frequent reassignments, including more frequent use of large processes, however it's intrinsic design, as illustrated in Figure 7.5, obfuscates deeper analysis due to the mechanisms related to the changing of weights in the objective function causing the cost to significantly increase in regions corresponding to high activity in reassignments of large processes in Figure 7.2. The vertical dashed blue lines represent changes in the neighborhood and illustrates how the adjusted weights max out at the half-way point between two adjacent neighborhoods, from which the algorithm beings the optimization on the modified objective function leading to the eventual downtrend. Also, based on the reassignments of VNS, it does not provide any indications of prioritization towards larger processes. On the other hand, CP-LNS reassignments in Figure 7.3 do show the largest process movements at the very start of the optimization, and overall higher average process size in the earlier stages with occasional BPRs throughout the optimization lifecycle, indicating

some level of prioritization towards larger processes.


Figure 7.6: Comparison of reassignment distribution vs. Process Sizes

Figure 7.6 compares the distribution of reassignments based on process size across algorithms, revealing that both VNS and CP-LNS tend to commit more frequent reassignments of larger processes compared to HC-LNSHC which has only single process reassignment of beyond 3e6 size and could indicate some missed potential for improvement for the HC-LNSHC algorithm. However, it is important to note that both VNS and CP-LNS algorithms reassign a significant number of small processes, which could be contributing to their overall performance and computational cost. Of the two, VNS performs considerably more BPRs than CP-LNS and not only that, it also presents a much more balanced distribution across the board for all process sizes and especially in the region of mid-sized processes (2e6 to 3e6), where both CP-LNS and HC-LNSHC algorithms are lacking in comparison and could be losing on a lot of potential gains. This ability to work with processes of any size could be attributed to the design of

the VNS algorithm as compared to LNS-based designs, providing it with more flexibility in exploration, and may have been the factor a key factor in VNS's overall performance.

Given the promising results of the HC-LNSHC algorithm the analysis extends to find more patterns to gain a more complete understanding of potential heuristics and build more on the knowledge towards RQ3. Having explored the reassignment patterns of downtrends and spike cycles, the next stage investigates the previously mentioned "critical threshold" point upon which the algorithm switches from HC to MIP-LNS. Figure A.1 shows the investigation using the Improvement metric (Difference in Objective Cost across reassignments) to more closely align with the actual underlying heuristic used within the algorithm. The subplots present the reassignment regions in occurs the HC to LNS switch, and further segmentation of one of those regions into even small parts is shown in Figure A.2 that present an attempt to identify some characteristic that could be used as potential trigger threshold for insights into building heuristics, such as "if improvement over region X is less than Y, use LNS". However, no potential criteria was identified - and specifically the opposite was found in this case, where regions of lesser improvement did not result in the falllback of LNS, but instead following regions showing higher improvements while still under HC ended up triggering the algorithm to switch to LNS.

Overall, this analysis provides a comprehensive review of the reassignments made by each algorithm, highlighting their respective optimization processes and the sizes of the processes being reassigned over time. Specifically, the HC-LNSHC by [13] has shown to be especially useful for understanding the dynamics of process reassignments and providing some guidance on building heuristics, following the general idea of starting largest and incrementally selecting smaller processes over the iterations. It is unfortunate a characteristic was not identified for the

switch point as that would enable the heuristic to have continuation into the next cycle. Additionally, the benefits of VNS were identified in the process size distribution over reassignments, where it was capable to extract potential improvement from regions of processes sizes that the other two algorithms were not.

# Chapter 8

# Conclusion

The primary of objectives of this reports to identify state-of-the-art optimization algorithms, implementation of a system to enable tracking of reassignments, as well as an analysis into the reassignments of best performing algorithms was achieved. Unfortunately, the heuristic to full complete RQ3 was not full achieved, however, the findings provide a solid foundation for future work to build upon. The main challenge with the available solutions stems from the lack of ability to purely evaluate the approaches based on their design due to the lack of standardization in technologies and tools, that would provide even grounds for evaluation. This primarly comes from the fact all the algorithms are implemented in different programming languages such as Java, C++, and C. The issues is that the JVM comes with an overhead as well as the garbage collector - whereas, the other two approaches had manually managed memory. This difference in technologies could have potentially significant impact on the end results, so it is hard to evaluate whether an approach is better from another in situations where a supoptimal algorithm could outperforms another purely because it was compiled to a native binary for the specific CPU architecture in runs on and is compared to algorithms executed from a virtual machine. However, the findings from this report provide

valuable insights into the reassignment behavior of each algorithm, which can inform future research and development efforts. By understanding the specific characteristics of each approach, it may be possible to develop more effective strategies to target specific weakness points of the analyzed algorithms to produce a more balanced approach. Additionally, the identification of key patterns and trends in the reassignments can help guide the design of new heuristics better suited to address domain-specific challenges within the optimization process.

# References


[1] T. Dillon, C. Wu, and E. Chang, "Cloud Computing: Issues and Challenges," in *2010 24th IEEE International Conference on Advanced Information Networking and Applications*. Perth, Australia: IEEE, 2010, pp. 27–33. 1, 2

[2] S. Bharany, S. Sharma, O. I. Khalaf, G. M. Abdulsahib, A. S. Al Humaimeedy, T. H. H. Aldhyani, M. Maashi, and H. Alkahtani, "A Systematic Survey on Energy-Efficient Techniques in Sustainable Cloud Computing," *Sustainability*, vol. 14, no. 10, p. 6256, May 2022. 1

[3] E. Masanet, A. Shehabi, N. Lei, S. Smith, and J. Koomey, "Recalibrating global data center energy-use estimates," *Science*, vol. 367, no. 6481, pp. 984–986, Feb. 2020. 1, 2

[4] A. Beloglazov and R. Buyya, "Energy Efficient Resource Management in Virtualized Cloud Data Centers," in *2010 10th IEEE/ACM International Conference on Cluster, Cloud and Grid Computing*. Melbourne, Australia: IEEE, 2010, pp. 826–831. 1

[5] A. Wreford, C. Saidu, M. Modibbo, and A. La'aro, "Methodological Approaches Used For the Google Machine Reassignment Problem." *Aligarh Journal of Statistics*, vol. 44, p. 25, 2024. 6, 8

[6] H. Murat Afsar, C. Artigues, E. Bourreau, and S. Kedad-Sidhoum, "Machine reassignment problem: The ROADEF/EURO challenge 2012," *Annals of Operations Research*, vol. 242, no. 1, pp. 1–17, Jul. 2016. 7, 14

[7] M. Pirlot, "General local search methods," *European Journal of Operational Research*, vol. 92, no. 3, pp. 493–511, Aug. 1996. 8

[8] C. Voudouris, E. P. Tsang, and A. Alsheddy, "Guided Local Search," in *Handbook of Metaheuristics*, M. Gendreau and J.-Y. Potvin, Eds. Boston, MA: Springer US, 2010, vol. 146, pp. 321–361. 10

[9] D. Pisinger and S. Ropke, "Large Neighborhood Search," in *Handbook of Metaheuristics*, M. Gendreau and J.-Y. Potvin, Eds. Cham: Springer International Publishing, 2019, vol. 272, pp. 99–127. 10

[10] P. Hansen, N. Mladenović, and J. A. Moreno Pérez, "Variable neighbourhood search: Methods and applications," *Annals of Operations Research*, vol. 175, no. 1, pp. 367–407, Mar. 2010. 10

[11] H. Gavranović, M. Buljubašić, and E. Demirović, "Variable Neighborhood Search for Google Machine Reassignment problem," *Electronic Notes in Discrete Mathematics*, vol. 39, pp. 209–216, Dec. 2012. 12, 21

[12] D. Mehta, B. O'Sullivan, and H. Simonis, "Comparing Solution Methods for the Machine Reassignment Problem," in *Principles and Practice of Constraint Programming*, M. Milano, Ed. Berlin, Heidelberg: Springer Berlin Heidelberg, 2012, vol. 7514, pp. 782–797. 13, 21, 22

[13] W. Jaśkowski, M. Szubert, and P. Gawron, "A hybrid MIP-based large neighborhood search heuristic for solving the machine reassignment problem," *Annals of Operations Research*, vol. 242, no. 1, pp. 33–62, Jul. 2016. 13, 21, 33, 37

[14] R. Hoffmann, M.-C. Riff, E. Montero, and N. Rojas, "Google challenge: A hyperheuristic for the Machine Reassignment problem," in *2015 IEEE Congress on Evolutionary Computation (CEC)*. Sendai, Japan: IEEE, May 2015, pp. 846–853. 13

[15] R. Lopes, V. W. Morais, T. F. Noronha, and V. A. Souza, "Heuristics and matheuristics for a real-life machine reassignment problem," *International Transactions in Operational Research*, vol. 22, no. 1, pp. 77–95, Jan. 2015. 13

[16] R. Masson, T. Vidal, J. Michallet, P. H. V. Penna, V. Petrucci, A. Subramanian, and H. Dubedout, "An iterated local search heuristic for multi-capacity bin packing and machine reassignment problems," *Expert Systems with Applications*, vol. 40, no. 13, pp. 5266–5275, Oct. 2013. 13

[17] M. Gabay and S. Zaourar, "Vector bin packing with heterogeneous bins: Application to the machine reassignment problem," *Annals of Operations Research*, vol. 242, no. 1, pp. 161–194, Jul. 2016. 13

[18] A. Turky, N. R. Sabar, and A. Song, "Neighbourhood Analysis: A Case Study on Google Machine Reassignment Problem," in *Artificial Life and Computational Intelligence*, M. Wagner, X. Li, and T. Hendtlass, Eds. Cham: Springer International Publishing, 2017, vol. 10142, pp. 228–237. 14

[19] ——, "Cooperative evolutionary heterogeneous simulated annealing algorithm for google machine reassignment problem," *Genetic Programming and Evolvable Machines*, vol. 19, no. 1-2, pp. 183–210, Jun. 2018. 14

[20] D. Canales, N. Rojas-Morales, and M.-C. Riff, "A Survey and a Classification of Recent Approaches to Solve the Google Machine Reassignment Problem,"

*IEEE access : practical innovations, open solutions*, vol. 8, pp. 88 815–88 829, 2020. 14

# Appendix A

# Additional Results of HC-LNSHC Analysis

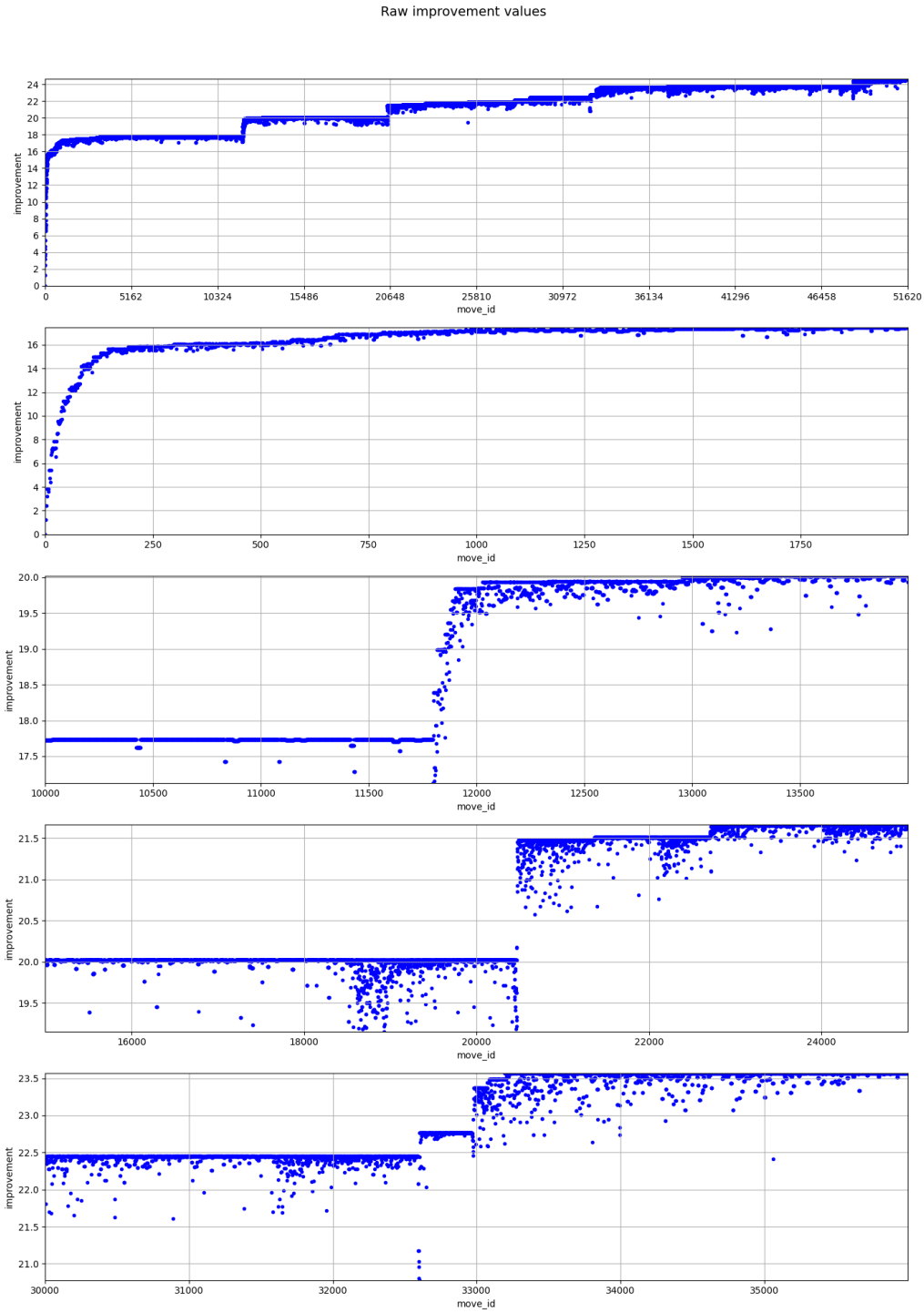


Figure A.1: Improvement Segments of HC-LNSHC Algorithm

Linear Regression on Improvement